CHAPTER 09

# Lambda and API Gateway

Serverless · Function URLs · API Gateway · Event Sources · SnapStart · ShopFlow



## 09-01 What is Serverless? Concepts and Use Cases

### ■ AWS Lambda

Lambda runs your code without you managing servers. You upload a function, AWS runs it when triggered, charges only for the milliseconds it executes, and scales automatically from 0 to 10,000 concurrent executions. No EC2 instances to patch, no containers to configure.

Example: ShopFlow uses Lambda for: order notifications (triggered by SQS), product image resizing (S3 trigger), search indexing (DynamoDB Streams trigger), and promotional email batches (scheduled EventBridge trigger).

When should you choose Lambda over ECS Fargate? The answer depends on your workload characteristics:

| Characteristic | Choose Lambda | Choose ECS Fargate |
|---|---|---|
| **Execution duration** | < 15 minutes per request | Long-running (hours, days) |
| **Traffic pattern** | Spiky / unpredictable (scales to zero) | Steady baseline traffic |
| **Cold start tolerance** | < 1s acceptable (or use SnapStart) | Zero cold start tolerance |
| **Memory needs** | Up to 10 GB | Up to 120 GB per task |
| **Concurrency** | Up to 10,000 simultaneous | Fixed task count (manual scaling) |
| **Cost model** | Pay per 100ms execution | Pay per vCPU/memory per hour |
| **ShopFlow usage** | Async processing, notifications, cron | Product API, cart API, checkout |

**INFO Lambda Pricing Example**

ShopFlow processes 500,000 order notifications/month via Lambda. Each runs 800ms using 256MB memory. Cost: 500,000 x 0.8s x 256MB/1024 = 100,000 GB-seconds. AWS Free Tier includes 400,000 GB-seconds/month. Above that: $0.0000166667 per GB-second. Monthly Lambda cost for notifications: ~$0. Even at 10x scale (5M notifications), cost is just $0.83/month.

## 09-02 Create Your First Lambda Function — Console Walkthrough

Let us create a Lambda function in the AWS Console. This hello world Lambda will later become the foundation for ShopFlow order notification handler.

### ■ Create a Lambda Function

1. Open the AWS Console -> type "Lambda" in the search bar -> click Lambda
2. Click the orange "Create function" button (top right)
3. Select "Author from scratch" (default)
4. Function name: shopflow-order-notification
5. Runtime: Java 21 (scroll down in the dropdown)
6. Architecture: arm64 (Graviton — 20% cheaper, same performance)
7. Execution role: Choose "Create a new role with basic Lambda permissions"
8. Click "Create function" — wait ~5 seconds
9. In the Code tab: upload your JAR via "Upload from" -> ".zip or .jar file"
10. Handler: com.shopflow.lambda.OrderNotificationHandler::handleRequest
11. Click "Deploy" to save the handler configuration

■ aws › Lambda > Functions > shopflow-order-notification

## Function overview

### ■ Function details

■ Function ARN: arn:aws:lambda:us-east-1:123456789:function:shopflow-order-notification

■ Runtime: Java 21 | Architecture: arm64 | Last modified: just now

### ■ Runtime settings

Handler

com.shopflow.lambda.OrderNotificationHandler::handleRequest

Memory (MB)

512 ■

Timeout

30 sec ■

### ■ Triggers

■ No triggers configured yet — we will add an SQS trigger in section 09-05

■ Function created successfully

Save

---

## 09-03 Lambda Handler — Java 21 with SnapStart

Java 21 on Lambda requires a handler class implementing RequestHandler. The cold start problem (JVM initialization) is solved by SnapStart — AWS takes a snapshot of the initialized JVM and restores it instead of starting from scratch, reducing cold starts from ~3 seconds to ~200ms.

```
// OrderNotificationHandler.java public class OrderNotificationHandler implements
RequestHandler { // SnapStart: initialized ONCE at snapshot time, reused on every invocation
private final SnsClient sns = SnsClient.builder() .region(Region.US_EAST_1).build(); private
final String topicArn = System.getenv("ORDER_NOTIFICATION_TOPIC_ARN"); @Override public Void
handleRequest(SQSEvent event, Context context) { for (SQSEvent.SQSMessage msg :
event.getRecords()) { OrderEvent order = parseOrder(msg.getBody());
sns.publish(PublishRequest.builder() .topicArn(topicArn) .subject("Order Confirmed: #" +
order.orderId()) .message(buildMessage(order)) .messageAttributes(Map.of( "orderStatus",
MessageAttributeValue.builder() .dataType("String") .stringValue(order.status()).build()))
.build()); context.getLogger().log("Notified order: " + order.orderId()); } return null; } }
```

**SNAP Enable SnapStart**

Lambda console: Configuration tab -> General configuration -> Edit -> SnapStart -> select "PublishedVersions" -> Save. Then publish a version: Actions -> Publish new version. SnapStart only applies to published versions, not $LATEST. Create an alias pointing to the published version for production.

---

## 09-04 API Gateway — REST API vs HTTP API vs WebSocket

### ■ Amazon API Gateway

API Gateway sits in front of your Lambda functions and handles HTTP routing, authentication, throttling, and request/response transformation. It turns your Lambda function into a real HTTPS endpoint accessible from the internet.

Example: ShopFlow uses HTTP API (v2) for its public product search endpoint backed by Lambda. HTTP API is 70% cheaper than REST API and has lower latency — ideal for simple proxy-to-Lambda patterns.

| Feature | REST API (v1) | HTTP API (v2) | WebSocket API |
|---|---|---|---|
| **Use case** | Complex transforms, SOAP | Simple Lambda proxy | Real-time bidirectional |
| **Latency** | ~6ms overhead | ~1ms overhead | Persistent connection |
| **Cost per million** | $3.50 | $1.00 | $1.00 + $0.25/M msgs |
| **Auth options** | Cognito, Lambda, IAM, key | Cognito, Lambda, IAM, JWT | Lambda authorizer only |
| **Request validation** | Built-in model validation | Limited | N/A |
| **VPC integration** | VPC Link (NLB) | VPC Link (ALB/NLB/ECS) | N/A |
| **Usage plans** | Yes (rate limit per API key) | No | N/A |
| **ShopFlow usage** | Internal admin APIs | Public search + product APIs | Order status updates |

---

## 09-05 Create an HTTP API in the Console

We will create an HTTP API that routes GET /products/search to the ShopFlow search Lambda — the most common beginner pattern: API Gateway -> Lambda.

| ■ Create HTTP API Gateway |
| --- |
| 1. AWS Console -> search "API Gateway" -> click API Gateway |
| 2. Click "Create API" |
| 3. Choose "HTTP API" -> click Build |
| 4. Integrations: click "Add integration" |
| 5. Integration type: Lambda -> select function: shopflow-product-search |
| 6. API name: shopflow-product-api -> click Next |
| 7. Configure routes: Method=GET, Resource path=/products/search, Integration=shopflow-product-search |
| 8. Click "Next" -> review stages ($default stage, auto-deploy enabled) |
| 9. Click "Create" |
| 10. Copy the Invoke URL: https://abc123.execute-api.us-east-1.amazonaws.com |
| 11. Test: curl "https://abc123.execute-api.us-east-1.amazonaws.com/products/search?q=shoes" |

■ aws › API Gateway > shopflow-product-api > Routes

## Routes

| ■ shopflow-product-api — HTTP API |
| --- |
| ■ Invoke URL: https://abc123def.execute-api.us-east-1.amazonaws.com |
| **■ Routes** |
| ■ Route: GET /products/search -> shopflow-product-search (Lambda) |
| ■ Route: GET /products/{id} -> shopflow-product-detail (Lambda) |
| ■ Route: POST /orders -> ECS Fargate VPC Link |
| ■ Auto-deploy enabled: changes deploy automatically to $default stage |
| ■ API deployed and accepting traffic |

---

**09-06** **Lambda + API Gateway with AWS CDK**

The CDK approach defines Lambda functions, SQS triggers, and API Gateway routes in code — version-controlled, repeatable, and reviewable in pull requests.

```
// ShopFlowLambdaStack.java (CDK) public class ShopFlowLambdaStack extends Stack { public
ShopFlowLambdaStack(App scope, String id, StackProps props) { super(scope, id, props); // 1.
Order Notification Lambda (SQS-triggered) Function orderFn = Function.Builder.create(this,
"OrderNotification") .functionName("shopflow-order-notification") .runtime(Runtime.JAVA_21)
.architecture(Architecture.ARM_64) // Graviton — 20% cheaper
.handler("com.shopflow.lambda.OrderNotificationHandler::handleRequest")
.code(Code.fromAsset("target/shopflow-lambda.jar")) .memorySize(512)
.timeout(Duration.seconds(30)) .snapStart(SnapStartConf.ON_PUBLISHED_VERSIONS)
.environment(Map.of("ORDER_NOTIFICATION_TOPIC_ARN", orderTopic.getTopicArn())) .build(); // 2.
SQS Event Source (SQS -> Lambda)
orderFn.addEventSource(SqsEventSource.Builder.create(orderQueue) .batchSize(10)
.maxBatchingWindow(Duration.seconds(5)) .reportBatchItemFailures(true) .build()); // 3. Product
Search Lambda + HTTP API Function searchFn = Function.Builder.create(this, "ProductSearch")
.runtime(Runtime.JAVA_21).architecture(Architecture.ARM_64)
```

```
.memorySize(1024).timeout(Duration.seconds(10)).build(); HttpApi httpApi =
HttpApi.Builder.create(this, "ProductApi") .apiName("shopflow-product-api").build();
httpApi.addRoutes(AddRoutesOptions.builder()
.path("/products/search").methods(List.of(HttpMethod.GET)) .integration(new
HttpLambdaIntegration("SearchInt", searchFn)) .build()); } }
```

## 09-07 Lambda Event Sources — How Triggers Work

Lambda functions are triggered by events from other AWS services. Each source has different batch sizes, retry behavior, and error handling semantics.

| Event Source | Trigger Model | ShopFlow Use Case | Key Config |
|---|---|---|---|
| **SQS Standard** | Lambda polls; batch up to 10,000 | Order notifications, email send | batchSize=10, visibility timeout = 6x Lambda timeo |
| **SQS FIFO** | Ordered delivery per MessageGroupId | Inventory updates (ordered) | Max 10 concurrent Lambdas per queue |
| **S3 PUT/DELETE** | Push: S3 calls Lambda on object event | Product image resizing | Filter by prefix (images/) and suffix (.jpg) |
| **DynamoDB Streams** | Lambda polls; ordered per partition | Search index updates on product change | startingPosition=LATEST, batchSize=100 |
| **EventBridge cron** | Scheduled cron expression | Daily report emails at 8 AM UTC | Disable rule to pause schedule |
| **SNS** | Push: SNS delivers to Lambda | Broadcast order events | DLQ on Lambda (not SNS) for retry |
| **API Gateway HTTP** | Sync: API waits for Lambda response | Product search, price check | 29s max timeout — API Gateway limit |
| **Kinesis** | Lambda polls stream shard | Real-time clickstream analytics | Enhanced fan-out for low latency |

## 09-08 Lambda Layers — Shared Dependencies

Lambda Layers let you share common code across multiple functions without bundling it into each deployment package. For ShopFlow, we use a layer for AWS SDK clients and logging — reducing each function JAR from 45MB to 8MB.

```
// Create a Lambda Layer via CDK LayerVersion commonLayer = LayerVersion.Builder .create(this,
"ShopFlowCommonLayer") .layerVersionName("shopflow-common-v1") .description("AWS SDK clients,
logging, common utilities") .code(Code.fromAsset("target/shopflow-common-layer.zip"))
.compatibleRuntimes(List.of(Runtime.JAVA_21))
.compatibleArchitectures(List.of(Architecture.ARM_64)) .build(); // Attach layer to function —
max 5 layers per function Function myFn = Function.Builder.create(this, "MyFn")
.layers(List.of(commonLayer)) // extracted to /opt/ .build();
```

| Layer Best Practice | Details |
|---|---|
| **One layer per concern** | Separate: aws-sdk layer, logging layer, utils layer — not one giant layer |
| **Version pinning** | Reference layer by ARN+version; never use "latest" in production |
| **Max 5 layers** | 250MB total unzipped across all layers + function code |
| **Public layers** | AWS publishes managed layers (Parameters and Secrets Lambda Extension) |
| **Layer for Java** | Put JARs in /java/lib/; Lambda adds /opt/java/lib to classpath automatically |

## 09-09 Monitoring Lambda — CloudWatch and X-Ray

Lambda automatically sends metrics to CloudWatch. The four metrics you must monitor in production: Errors, Duration, Throttles, and ConcurrentExecutions. Set alarms before going live.

■ aws › CloudWatch > Alarms > Create alarm

## Create CloudWatch Alarm for Lambda Errors

### ■ Step 1: Select metric

Namespace

AWS/Lambda ■

Metric name

Errors ■

FunctionName

shopflow-order-notification

Statistic

Sum ■

Period

1 minute ■

### ■ Step 2: Conditions

Threshold type

S t a t i c

Whenever Errors is...

G r e a t e r t h a n

Threshold value

5

### ■ Step 3: Actions

Alarm state trigger

In alarm ■

SNS topic

shopflow-ops-alerts ■

■ Alarm will trigger if Lambda errors exceed 5 per minute

**Create alarm**

| Metric | Alarm Threshold | What It Means |
|---|---|---|
| **Errors** | > 5 per minute | Function throwing exceptions; check logs immediately |
| **Duration P99** | > 25,000ms (25s) | Approaching 29s API Gateway timeout; optimize or go async |
| **Throttles** | > 0 per minute | Concurrency limit hit; request limit increase |
| **ConcurrentExecutions** | > 800 (of 1,000 default) | Approaching account concurrency limit |
| **Iterator Age (stream)** | > 60,000ms (1 minute) | Lambda falling behind on stream processing |

These practices come from running Lambda in production at scale. Beginners commonly skip all of them and then wonder why their functions fail mysteriously.

✓ Initialize SDK clients as static class fields outside the handler. They are created ONCE during Lambda init, snapshotted by SnapStart, and reused across invocations. Initializing inside handleRequest re-creates the TLS connection on every invocation.

■ Do not store state in static fields between invocations. Lambda containers are reused but cannot be relied upon. Session state belongs in ElastiCache Redis; persistent state in DynamoDB.

| Practice | Correct Approach |
|---|---|
| **Dead Letter Queue** | Every async Lambda must have a DLQ (SQS) for failed events after all retries |
| **Reserved concurrency** | Set reserved concurrency to prevent one function starving others |
| **Secrets** | Use Secrets Manager — never hardcode secrets in environment variables |
| **Idempotency** | Lambda may invoke twice (at-least-once). Use idempotency keys in DynamoDB to prevent duplicat |
| **Timeout** | Set timeout to 2x expected P99 duration, not the maximum 15 minutes |
| **Memory sizing** | Use Lambda Power Tuning tool — more memory = more CPU = often cheaper overall |

## 09-11 API Gateway Authentication with Cognito JWT

ShopFlow product search is public (no auth). The order history API requires authentication via Cognito JWT authorizer: the client sends a Bearer token, API Gateway validates it, and only invokes Lambda if the token is valid.

### ■ Add Cognito JWT Authorizer to HTTP API

1. API Gateway Console -> select shopflow-product-api
2. Left sidebar -> Authorization -> click "Add authorizer"
3. Authorizer type: JWT
4. Name: shopflow-cognito-authorizer
5. Identity source: $request.header.Authorization
6. Issuer URL: https://cognito-idp.us-east-1.amazonaws.com/us-east-1_XXXXX
7. (Get Pool ID: Cognito Console -> User Pools -> your pool -> Pool ID)
8. Audience: your Cognito App Client ID
9. Click "Create"
10. Routes -> POST /orders -> Attach authorizer -> shopflow-cognito-authorizer
11. Test: call POST /orders without token -> 401 Unauthorized

```
// Lambda handler — extract user from validated JWT claims public class OrderHandler implements RequestHandler { @Override public APIGatewayV2HTTPResponse handleRequest( APIGatewayV2HTTPEvent event, Context ctx) { // API Gateway already validated the JWT; claims are in the event Map claims = event.getRequestContext() .getAuthorizer().getJwt().getClaims(); String userId = (String) claims.get("sub"); // Cognito user ID String email = (String) claims.get("email"); // user email return processOrder(userId, email, event.getBody()); } }
```

## 09-12 Lambda Concurrency — Scaling and Throttling

Lambda scales by running more concurrent executions. Understanding the concurrency model prevents a common production incident: one function consuming all account concurrency and throttling everything else.

| Concurrency Type | How It Works | When to Use | ShopFlow Setting |
|---|---|---|---|
| **Unreserved (default)** | Shares the account-wide pool (1,000 default) | Most functions | Image resize Lambda: unreserved |
| **Reserved concurrency** | Guarantees N executions; no more, no less | Critical functions that must not starve others | Order notification: reserved=200 |
| **Provisioned concurrency** | Pre-warms N environments for zero cold starts | Latency-critical sync APIs (SnapStart is preferred for Java) | Product search: SnapStart instead |

**WARN Account Concurrency Limit**

Default Lambda concurrency limit is 1,000 per account per region. During Black Friday, request an increase to 10,000 via AWS Service Quotas (takes 1-2 business days — do this weeks in advance). Set reserved concurrency on critical functions to guarantee their share of the pool.

---

## 09-13 Lambda Function URLs — API Gateway Alternative

Function URLs give a Lambda function its own HTTPS endpoint without needing API Gateway. No routing, no stages, no extra cost. Ideal for webhooks where you only need one endpoint per function.

```
// CDK: Add Function URL to Lambda FunctionUrl webhookUrl = paymentFn.addFunctionUrl( FunctionUrlOptions.builder() .authType(FunctionUrlAuthType.NONE) // public — Stripe signs requests .cors(FunctionUrlCorsOptions.builder() .allowedOrigins(List.of("https://shopflow.com")) .allowedMethods(List.of(HttpMethod.POST)) .build()) .build()); // URL: https://abc123.lambda-url.us-east-1.on.aws/ // Configure this in Stripe Dashboard -> Webhooks -> Endpoint URL
```

| Feature | Function URL | API Gateway HTTP API |
|---|---|---|
| **Cost** | Free (pay only for Lambda invocations) | $1.00 per million requests |
| **Custom domain** | Not supported | Yes (via ACM + Route 53) |
| **Request routing** | One function per URL | Multiple routes -> multiple functions |
| **Auth options** | IAM_AUTH or NONE | JWT, Cognito, Lambda authorizer, IAM |
| **Best for** | Webhooks, internal tools, single-function APIs | Public APIs, multi-route apps |
| **ShopFlow usage** | Stripe payment webhook receiver | Product search, order APIs |

---

## 09-14 Optimizing Lambda Memory with Power Tuning

Counter-intuitive: increasing Lambda memory often decreases cost. More memory = more CPU = faster execution = less billed duration. The Lambda Power Tuning tool finds the optimal setting automatically.

| Memory | Duration (P50) | Cost per 1M invocations | Recommendation |
|---|---|---|---|
| **128 MB** | 4,200ms | $0.69 | Too slow — Java class loading starved for CPU |
| **256 MB** | 2,100ms | $0.35 | Still slow; timeout risk under load |
| **512 MB** | 850ms | $0.28 | Acceptable; most teams stop here |
| **1,024 MB** | 430ms | $0.28 | Same cost as 512MB, 2x faster — sweet spot |
| **2,048 MB** | 230ms | $0.30 | Slightly more expensive, marginal gain |
| **3,008 MB** | 200ms | $0.37 | No significant gain; not recommended |

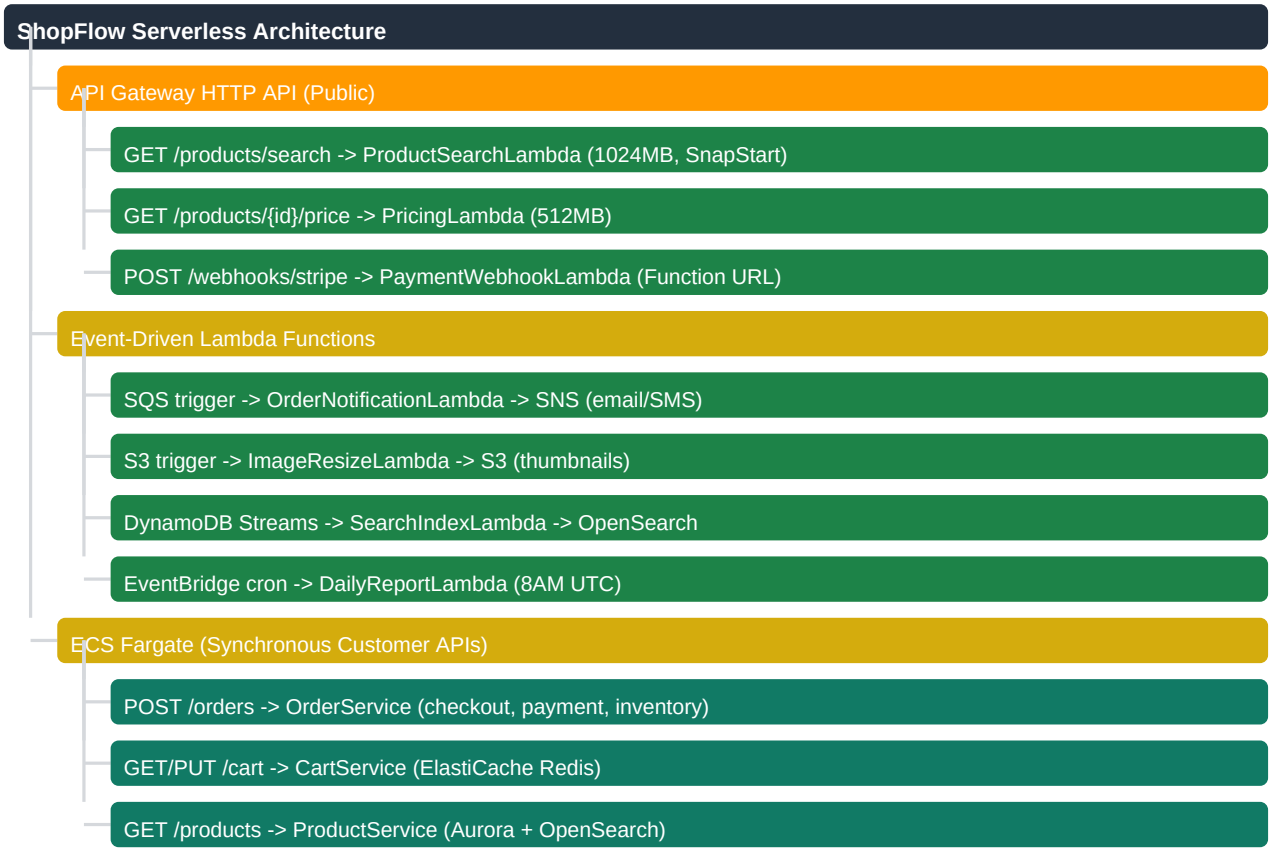
**TIP Run Power Tuning for ShopFlow**

Deploy the AWS Lambda Power Tuning SAR app (search "lambda-power-tuning" in Serverless Application Repository). It tests your function across memory settings from 128MB to 3,008MB and outputs a cost vs. speed chart. For ShopFlow Java 21 functions, 1,024MB is consistently the sweet spot — same cost as 512MB at 2x the speed.

---

## 09-15 ShopFlow Serverless Architecture — Complete Picture

Here is how Lambda and API Gateway fit into the complete ShopFlow architecture. Key principle: Lambda handles async and event-driven workloads; ECS Fargate handles synchronous customer-facing APIs.




| Lambda Function | Trigger | Memory | Timeout | Monthly Volume |
|---|---|---|---|---|
| **OrderNotificationLambda** | SQS (order-events queue) | 512 MB | 30s | 500K invocations |
| **ImageResizeLambda** | S3 PUT (images/ prefix) | 1,024 MB | 60s | 2M invocations |
| **SearchIndexLambda** | DynamoDB Streams (products table) | 512 MB | 15s | 100K invocations |
| **DailyReportLambda** | EventBridge cron (0 8 * * ?) | 256 MB | 300s | 30 invocations |
| **ProductSearchLambda** | API Gateway HTTP GET | 1,024 MB | 10s | 10M invocations |
| **PaymentWebhookLambda** | Function URL (Stripe) | 512 MB | 30s | 200K invocations |

## 09-16 Error Handling — Dead Letter Queues and Retries

Lambda retries failed async invocations twice (3 total attempts). After all retries, the event goes to the Dead Letter Queue. Without a DLQ, failed events are silently discarded.

```
// CDK: Configure DLQ on Lambda Queue dlq = Queue.Builder.create(this, "OrderNotificationDLQ")
.queueName("shopflow-order-notification-dlq") .retentionPeriod(Duration.days(14)) .build();
Function fn = Function.Builder.create(this, "OrderNotification") .deadLetterQueue(dlq)
.retryAttempts(2) // 0, 1, or 2 (default=2) .build(); // Alert on first failure
Alarm.Builder.create(this, "DLQAlarm") .metric(dlq.metricApproximateNumberOfMessagesVisible())
.threshold(1) .evaluationPeriods(1) .alarmName("shopflow-order-notification-dlq-not-empty")
.build() .addAlarmAction(new SnsAction(opsAlertTopic));
```

**INFO Re-processing DLQ Messages**

After fixing the bug: SQS Console -> select DLQ -> Start DLQ redrive -> Source: the original queue -> Start. Lambda will re-process all failed events. Set DLQ retention to 14 days to give yourself time to investigate before events expire.

## 09-17 Hands-On Lab — Build the ShopFlow Search API

Build the complete ShopFlow product search flow: API Gateway HTTP API -> Lambda -> in-memory product list (simulating OpenSearch). Time estimate: 45 minutes.

| ■ Lab: Build Product Search API |
|---|
| 1. Create Lambda function: shopflow-product-search (Java 21, arm64, 1024MB) |
| 2. Upload JAR with handler: com.shopflow.ProductSearchHandler::handleRequest |
| 3. Handler: parse "q" query param, filter in-memory product list, return JSON |
| 4. Test in Lambda console: Test tab -> create test event with queryStringParameters: {"q":"shoes"} |
| 5. Verify response: {"products":[...], "count": 3} |
| 6. Create HTTP API: shopflow-product-api |
| 7. Add route: GET /products/search -> shopflow-product-search |
| 8. Test: curl "https://.execute-api.us-east-1.amazonaws.com/products/search?q=shoes" |
| 9. Add CloudWatch alarm: Errors > 2 per minute -> SNS email alert |
| 10. Enable X-Ray tracing: Lambda -> Configuration -> Monitoring -> Active tracing ON |
| 11. View X-Ray trace: CloudWatch -> X-Ray traces -> filter by function name |
| 12. Cleanup: delete API, Lambda, CloudWatch alarm |

## 09-18 Common Questions — Lambda & API Gateway

| Question | Answer |
|---|---|
| **Can Lambda access resources in my VPC (RDS, ElastiCache)?** | Yes — configure Lambda VPC settings: VPC, subnets, security group. Cold starts are ~500ms slo |
| **What happens when Lambda times out?** | The invocation fails with Task timed out error. For async triggers (SQS), Lambda retries. For sync |
| **How do I pass secrets to Lambda?** | Use Secrets Manager or SSM Parameter Store. Read secrets at initialization time (static field), no |
| **How many Lambda functions should I have?** | One function per logical job. ShopFlow has 6 Lambda functions (see architecture table above). Av |
| **Does API Gateway cost money for development?** | HTTP API: $1.00 per million requests. AWS Free Tier: 1M free requests/month for 12 months. Lar |

## 09-19 Lambda Inside a VPC — Accessing Private Resources

By default Lambda runs in an AWS-managed VPC outside your ShopFlow VPC. To access private resources like Aurora, ElastiCache, or internal ECS services, you must attach Lambda to your VPC. This enables private IP connectivity but adds ~500ms to cold starts (mitigated by SnapStart).

■ aws › Lambda > Functions > shopflow-search-indexer > Configuration > VPC

## VPC Configuration

### ■ VPC settings

VPC

s h o p f l o w - v p c ( v p c - 0 a b c 1 2 3 4 5 d e f )

■ Subnets

Security groups

shopflow-lambda-sg (sg-0lambda12345) ■

### ■ Security group rules for shopflow-lambda-sg

■ Outbound: TCP 6379 -> shopflow-elasticache-sg (ElastiCache Redis)

■ Outbound: TCP 5432 -> shopflow-aurora-sg (Aurora PostgreSQL)

■ Outbound: TCP 443 -> 0.0.0.0/0 (AWS API calls via NAT Gateway)

■ No inbound rules needed — Lambda is always the initiator

■ VPC configuration saved. Cold start ~500ms; SnapStart reduces to ~200ms

Save

| VPC Lambda Consideration | Details |
|---|---|
| **Cold start increase** | VPC attachment adds ~500ms for ENI setup. SnapStart snapshots post-VPC-attach state, elim |
| **Subnet placement** | Place Lambda in app-tier subnets (private). Lambda needs outbound internet access (for Cloud |
| **Security group** | Lambda SG needs outbound rules only — one rule per resource type (Redis port 6379, Aurora |
| **DNS resolution** | VPC must have enableDnsResolution=true and enableDnsHostnames=true for SDK endpoint r |
| **Concurrent ENIs** | Each Lambda execution needs one ENI. 200 concurrent executions = up to 200 ENIs. Ensure s |

## 09-20 Lambda Destinations — Async Success and Failure Routing

Lambda Destinations let you route async invocation results — both successes and failures — to SQS, SNS, Lambda, or EventBridge without writing code. This is more flexible than DLQ, which only handles failures.

```
// CDK: Configure Lambda Destinations Function fn = Function.Builder.create(this, "OrderNotification") .onSuccess(new SqsDestination(successQueue)) // route successes to audit queue .onFailure(new SqsDestination(dlq)) // route failures to DLQ .build(); // Event payload sent to destination on SUCCESS: // { // "version": "1.0", // "requestContext": { "functionArn": "...", "condition": "Success" }, // "requestPayload": { ...original SQS event... }, // "responsePayload": null // Void handler returns null // } // Event payload on FAILURE: // { // "requestContext": { "condition": "RetriesExhausted", "functionArn": "..." }, // "requestPayload": { ...original event... }, // "responseContext": { "functionError": "Unhandled", // "statusCode": 200 } // }
```

| Routing Option | DLQ (Dead Letter Queue) | Lambda Destinations |
|---|---|---|
| **Captures failures** | Yes | Yes (onFailure) |
| **Captures successes** | No | Yes (onSuccess) — useful for audit |
| **Payload detail** | Original event only | Original event + response + context |
| **Supported targets** | SQS, SNS | SQS, SNS, Lambda, EventBridge |

| Routing Option | DLQ (Dead Letter Queue) | Lambda Destinations |
|---|---|---|
| **Use together** | Yes — DLQ for SQS trigger failures; Destinations for async failures | |

## 09-21 API Gateway Request Validation and Throttling

API Gateway can validate requests before they reach Lambda — rejecting malformed requests immediately and saving Lambda invocation costs. Throttling prevents runaway clients from generating surprise bills.

### ■ Configure API Gateway Throttling

1. API Gateway Console -> select shopflow-product-api
2. Left sidebar -> Stages -> $default
3. Default route throttling: Rate = 500 requests/second, Burst = 1,000
4. Click "Save changes"
5. Per-route override: Routes -> GET /products/search -> Route settings
6. Route throttling: Rate = 200 req/s, Burst = 500 (search is expensive)
7. Save — throttle applies immediately; excess requests receive 429 Too Many Requests

| Throttle Setting | Value | What Happens When Exceeded |
|---|---|---|
| **Default rate limit** | 500 req/s (account default) | 429 Too Many Requests returned by API Gateway (Lambda NOT inv |
| **Default burst limit** | 1,000 requests (token bucket) | Burst absorbs traffic spikes; bucket refills at rate limit speed |
| **Per-route: /products/search** | Rate=200, Burst=500 | Search route throttled independently — protects OpenSearch from o |
| **WAF rate rule (recommended)** | 1,000 req/5min per IP | Block IPs exceeding limit before they reach API Gateway (earlier, ch |

**INFO ShopFlow Rate Limiting Strategy**

Three layers of rate limiting: (1) WAF rate rule blocks IPs exceeding 1,000 requests per 5 minutes before they reach API Gateway. (2) API Gateway throttles at 500 req/s overall and 200 req/s for the expensive search route. (3) Lambda rate limiting in ElastiCache Redis (1,000 req/min per user ID) for authenticated endpoints. Layered defense prevents any single abusive client from impacting other users.

## 09-22 EventBridge Scheduled Lambda — Cron Jobs Without Servers

EventBridge replaces traditional cron jobs. Instead of maintaining an EC2 instance running cron, define a schedule rule and EventBridge invokes your Lambda function automatically. ShopFlow uses this for daily reports, cache warming, and weekly analytics.

```
// CDK: EventBridge Scheduled Lambda Function dailyReportFn = Function.Builder.create(this,
"DailyReport") .functionName("shopflow-daily-report") .runtime(Runtime.JAVA_21)
.architecture(Architecture.ARM_64) .memorySize(256) .timeout(Duration.minutes(5)) .build(); //
Run every day at 8:00 AM UTC Rule dailyRule = Rule.Builder.create(this, "DailyReportSchedule")
.ruleName("shopflow-daily-report-8am") .description("Trigger daily sales report generation")
.schedule(Schedule.cron(CronOptions.builder() .hour("8").minute("0") .build())) .build();
dailyRule.addTarget(new LambdaFunction(dailyReportFn, LambdaFunctionProps.builder()
.event(RuleTargetInput.fromObject(Map.of( "reportType", "daily-sales", "includeRefunds",
true))) .retryAttempts(2) .deadLetterQueue(reportDlq) .build()));
```

| ShopFlow Scheduled Lambda | Schedule | Purpose | Memory |
|---|---|---|---|
| **shopflow-daily-report** | 0 8 * * ? * (daily 8AM UTC) | Generate daily sales PDF, email to managers | 256 MB |
| **shopflow-cache-warmer** | 0/15 * * * ? * (every 15 min) | Pre-fetch top 100 products into Redis | 512 MB |

| ShopFlow Scheduled Lambda | Schedule | Purpose | Memory |
|---|---|---|---|
| **shopflow-weekly-analytics** | 0 6 ? * MON * (Mon 6AM UTC) | Aggregate weekly metrics into Dy | 512 MB |
| **shopflow-session-cleanup** | 0 2 * * ? * (daily 2AM UTC) | Delete expired sessions from Dynamo | 128 MB |
| **shopflow-image-optimizer** | 0 3 * * ? * (daily 3AM UTC) | Re-compress old product images, | 1,024 MB |

## 09-23 Cold Start Deep Dive — Why Java is Slow and How to Fix It

Cold starts happen when Lambda initializes a new execution environment. For Java, this involves JVM startup, class loading, Spring context initialization, and connection pool creation — all adding latency before your handler even runs.

| Phase | Without SnapStart | With SnapStart | Description |
|---|---|---|---|
| **JVM startup** | 150ms | 0ms (restored from snapshot) | JVM binary initialization |
| **Class loading** | 800ms | 0ms (restored) | Loading and verifying all classes |
| **SDK client init** | 400ms | 0ms (restored) | Creating TLS connections, loading credentials |
| **First invocation** | 1,350ms total | ~200ms (snapshot restore) | Full cold start vs. restore time |
| **Subsequent (warm)** | ~50ms | ~50ms | Handler execution only — same either way |

Additional cold start reduction strategies beyond SnapStart:

✓ Use arm64 (Graviton) architecture — Java class loading is ~15% faster on arm64 vs x86

✓ Set memory >= 1024MB for Java — more CPU reduces class loading and JVM startup time

✓ Minimize dependencies — each additional JAR on the classpath adds class loading time. Remove unused Spring Boot auto-configurations with @SpringBootApplication(exclude=[...])

✓ Keep initialization in static fields — everything initialized outside the handler is snapshotted. Everything inside runs per-invocation.

■ Do not use Spring Boot full context in Lambda — it loads hundreds of beans. Use lightweight initialization: just the SDKs you need, no Spring MVC, no JPA autoconfigure.

## 09-24 Lambda Security — IAM Roles, Resource Policies, and VPC

Every Lambda function runs with an IAM execution role. This role determines what AWS services the function can call. Follow least-privilege: grant only the specific actions on specific resources the function needs.

```
// CDK: Least-privilege IAM for OrderNotificationLambda // Grant ONLY: publish to the specific SNS topic // NOT: sns:* on * (common mistake — grants full SNS access) orderTopic.grantPublish(orderNotificationFn); // Grant ONLY: read/write to the specific DynamoDB table // NOT: dynamodb:* on * (would allow deleting any table) idempotencyTable.grantReadWriteData(orderNotificationFn); // Grant ONLY: read specific Secrets Manager secret Secret dbSecret = Secret.fromSecretNameV2(this, "DbSecret", "shopflow/production/aurora-password"); dbSecret.grantRead(orderNotificationFn); // Review what CDK generated: cdk synth | grep -A 30 "OrderNotification" // You will see a tightly scoped policy with specific ARNs
```

| Security Requirement | Implementation |
|---|---|
| **Execution role per function** | One unique IAM role per Lambda function — never share roles across functions with different p |
| **Secrets management** | Read from Secrets Manager at init time; cache in static field; never log secret values or put in e |
| **VPC for private resources** | Attach to VPC for Aurora/Redis access; security group allows only necessary outbound ports |
| **Input validation** | API Gateway model validation + Lambda-side validation — never trust event payload; validate |
| **Resource-based policy** | Restrict which services can invoke Lambda: api-gateway:* from your API ID, sqs:* from your qu |

## 09-25 Lambda Cost Optimization for ShopFlow

Lambda is already inexpensive but there are several patterns that can reduce costs further. The biggest lever is function memory tuning — a 10x difference between 128MB and 1024MB Java functions at the same price.

| Optimization | Approach | ShopFlow Savings |
|---|---|---|
| **Memory tuning** | Use Lambda Power Tuning to find cost-optimal memory. Java at 512MB often faster and cheaper than 256MB. | $200/month vs $400 at 128MB for 10M invocations |
| **Batch processing** | SQS trigger with batchSize=10 processes 10 messages per invocation. Reduces invocation count by 10x. | $0.02 vs $0.20 per million SQS messages |
| **arm64 architecture** | Graviton2 Lambda: 20% cheaper per GB-second, typically 10-19% better performance. | ~20% savings across all functions |
| **SnapStart** | Eliminates provisioned concurrency cost for Java functions. SnapStart is free and provides near-instant cold starts. | $130/month vs $0 for 10 provisioned instances |
| **S3 instead of layers** | Large static assets (ML models, reference data) in S3 rather than Lambda layers. Layers count toward 250MB unzipped limit. | Applicable for ShopFlow's ML recommendation model |

**INFO ShopFlow Lambda Monthly Cost Estimate**

At 1M MAU with current Lambda usage: OrderNotification (500K x 512MB x 30s max, actual avg 0.8s) = $0.83. ImageResize (2M x 1024MB x avg 2s) = $66. SearchIndex (100K x 512MB x avg 0.5s) = $0.04. ProductSearch (10M x 1024MB x avg 0.4s) = $66. Total Lambda cost: ~$133/month. Compare to ECS Fargate for the same workloads: estimated $400+/month at equivalent capacity. Lambda saves ~$267/month for async/event-driven workloads.

## 09-26 Black Friday Lambda Readiness Checklist

Black Friday generates 10x normal traffic for ShopFlow. Lambda scales automatically, but preparation prevents the non-obvious failure modes: concurrency limits, SQS queue depth spikes, cold starts under burst, and downstream service saturation.

| Checklist Item | Action Required | Status Check |
|---|---|---|
| **Concurrency limit** | Request increase to 10,000 via Service Quotas 2 weeks before event. Default 1,000 will throttle at ~1,000 si |
| **Reserved concurrency** | Set: OrderNotification=200, ProductSearch=500, ImageResize=100. Reserve 200 for other functions. |
| **SQS visibility timeout** | Set to 6x Lambda timeout. OrderNotification timeout=30s → SQS visibility=180s. Prevents duplicate p |
| **DLQ alarms** | Verify CloudWatch alarms on all DLQ queues alert ops@shopflow.com. Test by manually s |
| **SnapStart versions** | Publish new Lambda version and update alias to point to it. SnapStart only works on published version |
| **Load test** | Run Artillery/k6 load test at 2x expected peak. Verify auto-scaling, P99 latency, and DLQ empty after test. |
| **SQS queue depth** | Set CloudWatch alarm: SQS ApproximateNumberOfMessages > 5,000 for 5 minutes. Indicates consumers falling beh |
| **Dead letter routing** | Confirm all async Lambdas have onFailure destination or DLQ configured. Use Lambda Console > Configuration |

## 09-27 Orchestrating Lambda with Step Functions

When a business process requires multiple Lambda functions in sequence — with error handling, retries, and conditional branching — AWS Step Functions provides a visual workflow engine. ShopFlow uses Step Functions for the order fulfillment pipeline: validate order -> charge payment -> update inventory -> send notification -> generate invoice.

```
// CDK: Step Functions state machine for order fulfillment LambdaInvoke validateOrder =
LambdaInvoke.Builder .create(this, "ValidateOrder") .lambdaFunction(validateOrderFn)
.resultPath("$.validation") .build(); LambdaInvoke chargePayment = LambdaInvoke.Builder
.create(this, "ChargePayment") .lambdaFunction(chargePaymentFn) .resultPath("$.payment")
.build(); LambdaInvoke updateInventory = LambdaInvoke.Builder .create(this, "UpdateInventory")
.lambdaFunction(updateInventoryFn).build(); LambdaInvoke sendNotification =
LambdaInvoke.Builder .create(this, "SendNotification")
.lambdaFunction(orderNotificationFn).build(); // Chain: validate -> charge -> update inventory
-> notify // Each step retries on failure before moving to error handler Chain definition =
```

```
validateOrder .next(chargePayment) .next(updateInventory) .next(sendNotification);
StateMachine.Builder.create(this, "OrderFulfillment")
.stateMachineName("shopflow-order-fulfillment") .definition(definition)
.stateMachineType(StateMachineType.EXPRESS) // sync, < 5 min, cheaper .build();
```

| Workflow Feature | Without Step Functions | With Step Functions |
|---|---|---|
| **Error handling** | Each Lambda manually catches errors and retries; no state | Visual retry policies per state; catch blocks route to compensating tra |
| **Visibility** | No visibility into where an order is in the pipeline | Step Functions Console shows real-time execution graph with input/ |
| **Partial failures** | Hard to implement compensation (e.g., refund if inventory fails) | Catch blocks trigger compensating Lambdas automatically on inven |
| **Cost (Step Functions Express)** | N/A | $1.00 per 1M state transitions; 500K orders x 5 steps = 2.5M transiti |

## 09-BP Best Practices and Anti-Patterns

| Best Practice | Anti-Pattern to Avoid |
|---|---|
| **Initialize SDK clients as static class fields (SnapStart snapshots them)** | Creating S3Client/DynamoDbClient inside handleRequest — re-creates TLS |
| **Enable SnapStart on published versions for Java Lambda** | Leaving Java Lambda without SnapStart — 3-5s cold starts during low-traffic |
| **Set reserved concurrency on critical functions** | Leaving all functions unreserved — one spike can throttle all other functions |
| **Add DLQ to every async Lambda; alarm on DLQ depth > 0** | Skipping DLQ — failed events silently disappear with no alerting or retry path |
| **Use HTTP API (v2) for simple Lambda proxy — 70% cheaper** | Using REST API (v1) for basic Lambda integrations that need none of its extra |
| **Set API Gateway throttling: burst=1000, rate=500 req/s** | No throttling — a misconfigured client loop can generate massive Lambda cos |
| **Size Lambda memory with Power Tuning tool** | Leaving all functions at default 128MB — Java is CPU-bound; 128MB is 10x s |